FORMATION « PYTHON EN 24 HEURES » | SÉANCE 08

Classes et objets : encapsulation

Chapitres 15 et 16 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Passer du style procédural au style objet : la classe comme moule
- Écrire __init__ et comprendre le rôle de self
- Distinguer attributs d'instance et attributs de classe
- Protéger les données avec les conventions _ et __ et exposer via @property
- Utiliser @classmethod et @staticmethod à bon escient

Rappel de la séance 07

Jusqu'ici, les données (dict, list) et les fonctions vivaient séparément. L'objet réunit les deux dans une même entité.

- LEGB : où Python cherche les noms
- Closures et nonlocal : une fonction qui garde son contexte
- Décorateurs : emballer une fonction sans la modifier
- Chaînes : strip, lower, split, join et le chaînage
- Fichiers : with open, modes r, w, a

Du procédural à l'orienté objet

- Procédural : des données d'un côté, des fonctions de l'autre
- Objet : des données (**attributs**) et des comportements (**méthodes**) réunis
- La **classe** est le moule ; l'**objet** (ou instance) est le produit du moule
- Chaque instance garde ses propres valeurs
- Exemple : mille comptes bancaires, une seule classe Compte

```python
class Compte:
    def deposer(self, montant):
        self.solde += montant

compte_awa = Compte()
compte_awa.solde = 1000
compte_awa.deposer(500)
print(compte_awa.solde)    # 1500
```

`__init__` et self : la naissance d'un objet

```
class Compte:
    def __init__(self, titulaire, solde=0):
        self.titulaire = titulaire
        self.solde = solde

    def deposer(self, montant):
        self.solde += montant

compte = Compte("Awa", 1000)
compte.deposer(500)
print(compte.titulaire, compte.solde)
```

`__init__` est appelé automatiquement à la création. self désigne l'instance courante : compte.deposer(500) passe compte en self.

self, mode d'emploi

- Le premier paramètre de toute méthode d'instance est **self**
- ‘compte.deposer(500)‘ équivaut à ‘Compte.deposer(compte, 500)‘
- Oublier self dans la définition est l'erreur n°1 du débutant
- Les attributs naissent par affectation : ‘self.x = ...‘
- On ne passe **jamais** self soi même à l'appel

Attribut d'instance contre attribut de classe

- Attribut d'instance : défini dans __init__ via self, propre à chaque objet
- Attribut de classe : défini au niveau de la classe, **partagé** par toutes les instances
- Usage : constante commune, compteur d'instances
- Piège : réaffecter via self crée une copie d'instance qui **masque** l'attribut de classe

```
class Compte:
    banque = "UBA"          # attribut de classe

    def __init__(self, titulaire):
        self.titulaire = titulaire  # d'instance

a = Compte("Awa")
b = Compte("Kofi")
print(a.banque, b.banque)       # UBA UBA
Compte.banque = "Ecobank"
print(a.banque, b.banque)       # Ecobank Ecobank
```

Encapsulation : protéger l'intérieur

- Principe : le monde extérieur passe par des **méthodes**, pas par les attributs bruts
- Convention '_attribut' : usage interne, signal aux lecteurs
- Convention '__attribut' : Python déforme le nom (name mangling), accès direct bloqué
- Objectif : valider chaque modification au lieu de subir n'importe quelle valeur
- Un solde négatif saisi de l'extérieur ne doit jamais être possible

@property : l'attribut intelligent

```
class Compte:
    def __init__(self, titulaire, solde=0):
        self._solde = solde
    @property
    def solde(self):
        return self._solde

    @solde.setter
    def solde(self, valeur):
        if valeur < 0:
            raise ValueError("solde négatif interdit")
        self._solde = valeur
compte = Compte("Awa", 500)
compte.solde = 800      # passe par le setter
print(compte.solde)     # passe par le getter
```

Le code appelant écrit compte.solde comme un attribut simple, mais chaque accès passe par vos méthodes : la validation est invisible et garantie.

@classmethod et @staticmethod

```python
class Compte:
    nombre = 0
    def __init__(self):
        Compte.nombre += 1

    @classmethod
    def combien(cls):
        return cls.nombre

    @staticmethod
    def valide(montant):
        return isinstance(montant, (int, float)) and montant > 0
print(Compte.combien())   # 0
Compte(); Compte()
print(Compte.combien())   # 2
```

classmethod reçoit la classe (cls) : parfait pour les fabriques et compteurs. staticmethod ne reçoit rien : une fonction rangée dans la classe par convenance.

Fabrique via @classmethod

```
class Compte:
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire
        self.solde = solde

    @classmethod
    def creer_depuis_chaine(cls, texte):
        nom, montant = texte.split(":")
        return cls(nom, int(montant))

compte = Compte.creer_depuis_chaine("Awa:1500")
print(compte.titulaire, compte.solde)
```

La fabrique alternative : l'appelant écrit Compte.creer_depuis_chaine(« Awa:1500 ») et reçoit un objet prêt. cls désigne la classe, quelles que soient ses futures filles.

`__str__` : un objet qui se présente

```python
class Compte:
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire
        self.solde = solde

    def __str__(self):
        return f"Compte de {self.titulaire} : {self.solde} F"

compte = Compte("Awa", 1500)
print(compte)              # utilise __str__
```

print() cherche `__str__` ; sans lui, on voit l'adresse mémoire <__main__.Compte object at 0x...>. Nous approfondirons les méthodes spéciales à la séance 09.

Attention

Le piège de la mutable partagée

Un attribut de classe de type liste est partagé par toutes les instances : ‘Compte.historique.append(...)‘ écrit dans LA liste commune à tous les comptes. Règle : toute donnée propre à une instance naît dans __init__ via self. L’attribut de classe reste réservé aux constantes et aux compteurs.

À retenir

L'essentiel de la séance 08

La classe définit un moule : __init__ fabrique l'instance et self la désigne partout. Les attributs d'instance sont individuels, les attributs de classe partagés. L'encapsulation cache les données (_ ou __) et les expose via @property avec validation. @classmethod reçoit la classe, @staticmethod n'est qu'une fonction bien rangée.

Travaux pratiques (1 h) : Classes, objets et encapsulation

Cinq exercices pour fabriquer vos premières classes solides. Les exercices 3 à 5 préparent le grand mini projet bancaire de la séance 10 : gardez ce code sous la main.

- **Exercice 1 (Basique)** : Classe Point
- **Exercice 2 (Basique)** : Classe Livre
- **Exercice 3 (Intermédiaire)** : CompteSimple validé
- **Exercice 4 (Intermédiaire)** : Température avec @property
- **Exercice 5 (Défi)** : Playlist avec compteur de créations

Ouvrez le notebook « Seance_08_classes_et_encapsulation.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 09 : héritage, polymorphisme et méthodes spéciales. Vos classes vont se transmettre des comportements et apprendre à se comparer, à s'additionner et à se comporter comme des listes. Relisez `__str__` de cette séance : il devient `__repr__` et prend une autre dimension.